Shell Scripting Interview Questions & Answers

For DevOps Engineer, SRE, and Cloud Engineer Interviews

1. What is Shell Scripting?

Answer: Shell scripting is writing a series of Linux commands in a file so that they can be executed automatically.

DevOps use: Shell scripts are commonly used for deployment, backups, server maintenance, log processing, and automation.

2. What is a Shell?

Answer: A shell is a program that acts as an interface between the user and the operating system. Common shells are Bash, sh, Zsh, and Ksh. Bash is one of the most commonly used shells in Linux.

3. What is Bash?

Answer: Bash stands for **Bourne Again Shell**. It is a command-line shell and scripting language commonly used in Linux.

```
#!/bin/bash
echo "Hello World"
```

4. What is a shell script?

Answer: A shell script is a file containing a sequence of shell commands. Usually, it has the `.sh` extension.

5. What is #!/bin/bash?

Answer: `#!/bin/bash` is called a **shebang**. It tells the operating system which interpreter should be used to execute the script.

6. How do you create and execute a shell script?

Answer:

```
vim script.sh
chmod +x script.sh
./script.sh
```

Alternatively, run it with `bash script.sh`.

7. How do you create a variable in shell scripting?

Answer:

```
name="Suphiya"
echo $name
```

There should be no spaces around =.

8. What is the difference between $name and ${name}?

Answer: Both access a variable. `${name}` is useful when combining a variable with other characters, for example `echo "${name}_backup"`.

9. How do you take user input in shell scripting?

Answer: Use `read`.

```
echo "Enter your name:"
read name
echo "Hello $name"
```

10. What are environment variables?

Answer: Environment variables are variables available to processes running in the operating system. Examples include PATH, HOME, USER, and SHELL. Use `env` to view them.

11. What is $PATH?

Answer: PATH is an environment variable containing directories where Linux looks for executable commands. Check it with `echo $PATH`.

12. What are positional parameters?

Answer: Positional parameters allow us to pass arguments to a shell script. `$1` is the first argument and `$2` is the second argument.

13. What is $0?

Answer: `$0` represents the name of the script.

14. What are $1, $2, $3?

Answer: They represent the first, second, and third arguments passed to the script. For example, `./script.sh AWS Azure GCP`.

15. What is $#?

Answer: $# gives the number of arguments passed to the script.

16. What is $@?

Answer: $@ represents all arguments passed to the script.

17. What is $??

Answer: The special variable $? contains the exit status of the previous command. Generally, 0 means success and a non-zero value means failure. This is very important in DevOps automation.

18. How do you use an if condition?

Answer:

```
if [ "$1" == "start" ]
then
echo "Starting application"
else
echo "Invalid option"
fi
```

19. How do you check whether a file exists?

Answer:

```
if [ -f "test.txt" ]
then
echo "File exists"
else
echo "File does not exist"
fi
```

Common tests: -f file, -d directory, -e exists, -r readable, -w writable, -x executable.

20. How do you check whether a directory exists?

Answer:

```
if [ -d "/backup" ]
then
echo "Directory exists"
else
echo "Directory does not exist"
fi
```

21. What is a for loop?

Answer: A for loop executes commands repeatedly for a set of values.

```
for i in 1 2 3 4 5
do
echo $i
done
```

22. Give a practical DevOps example of a for loop.

Answer:

```
for server in server1 server2 server3
do
ping -c 1 $server
done
```

This can be useful for basic automation and health checks.

23. What is a while loop?

Answer:

```
count=1
while [ $count -le 5 ]
do
echo $count
count=$((count+1))
done
```

24. What is a function in shell scripting?

Answer: A function is a reusable block of commands.

```
backup()
{
echo "Taking backup"
}
backup
```

25. Why are functions useful in DevOps scripts?

Answer: Functions help organize large scripts into reusable sections such as install_dependencies, build_application, and deploy_application. This makes automation scripts easier to maintain.

26. How do you find a process?

Answer:

```
ps -ef
ps aux
ps -ef | grep nginx
```

27. How do you check CPU and memory usage?

Answer: Use `top` or `htop` for processes, `free -h` for memory, and `df -h` for disk.

28. How do you check disk usage of a directory?

Answer:

```
du -sh /var/log
du -sh /var/log/*
```

Useful when troubleshooting a server with low disk space.

29. How do you find files larger than 1 GB?

Answer:

```
find /var -type f -size +1G
```

30. How do you find files modified in the last 24 hours?

Answer:

```
find /var/log -type f -mtime -1
```

31. How do you search for a word in a log file?

Answer:

```
grep "ERROR" application.log
grep -i "error" application.log
```

32. How do you count the number of errors in a log file?

Answer:

```
grep -i "error" application.log | wc -l
```

grep searches, the pipe passes output, and wc -l counts lines.

33. How do you monitor a log file in real time?

Answer:
```
tail -f application.log
```

This is commonly used by DevOps and SRE engineers while troubleshooting.

34. What is the difference between grep, awk, and sed?

Answer:
grep → search/filter text.
awk → process and extract structured text.
sed → modify/replace text.

Remember: **grep** → **Find, awk** → **Extract/Process, sed** → **Modify.**

35. What is >?

Answer: > redirects output to a file and overwrites the file.
```
echo "Hello" > test.txt
```

36. What is >>?

Answer: >> redirects output and appends it to the file.
```
echo "Hello" >> test.txt
```

37. What is | pipe?

Answer: A pipe sends the output of one command as input to another command.
```
ps -ef | grep nginx
```

38. What is the difference between 2> and >?

Answer: > redirects standard output, while 2> redirects standard error.
```
command > output.txt
command 2> error.txt
```

39. Why are exit codes important in DevOps?

Answer: Exit codes tell automation tools whether a command succeeded or failed. Generally 0 means success and non-zero means failure. They are important in CI/CD pipelines.

40. What is set -e?

Answer: `set -e` tells Bash to exit when a command returns a non-zero status in situations covered by Bash's error handling rules. It can prevent a deployment script from continuing after an important failure.

41. What is set -x?

Answer: `set -x` prints commands as Bash executes them. It is useful for debugging scripts.

42. What is set -u?

Answer: `set -u` causes Bash to treat unset variables as an error in many contexts. It helps catch missing variables.

43. Write a script to check whether a service is running.

Answer:

```
#!/bin/bash

if systemctl is-active --quiet nginx
then
echo "Nginx is running"
else
echo "Nginx is not running"
fi
```

44. Write a script to check disk usage.

Answer:

```
#!/bin/bash

usage=$(df / | awk 'NR==2 {print $5}' | tr -d '%')

if [ "$usage" -gt 80 ]
then
echo "WARNING: Disk usage is above 80%"
else
echo "Disk usage is normal"
fi
```

45. How would you find the top 5 largest files?

Answer:

```
du -ah /var | sort -rh | head -5
```

du → disk usage; -a → files and directories; -h → human-readable; sort -rh → reverse human-readable order; head -5 → first five results.

46. How would you check whether a particular port is listening?

Answer:

```
ss -lntp
ss -lntp | grep :8080
```

47. How do you kill a process?

Answer: Find the process with `ps -ef | grep nginx`, then use `kill PID`. If it does not stop gracefully, `kill -9 PID` can be used. Prefer normal termination first.

48. How do you schedule a shell script?

Answer: Use cron. Edit with `crontab -e`.

Example: `0 2 * * * /home/user/backup.sh` runs every day at 2:00 AM.

49. What is a cron job?

Answer: A cron job is a scheduled task in Linux.

```
0 2 * * * /home/user/backup.sh
```

Format: minute hour day month weekday command

50. How do you make a script executable?

Answer:

```
chmod +x script.sh
./script.sh
```

■ Important DevOps/SRE Interview Questions

Beginner

1. What is shell scripting?
2. What is Bash?
3. What is a shebang?
4. How do you execute a script?
5. What are variables?
6. What are positional parameters?
7. What is $?
8. What is $#?
9. What is $@?
10. How do if conditions work?
11. How do for and while loops work?
12. What are functions?
13. What is grep?
14. What is awk?
15. What is sed?
16. What is a pipe?
17. What are > and >>?
18. What is cron?

Intermediate

1. How do you check disk usage?
2. How do you check CPU/memory?
3. How do you find large files?
4. How do you find files modified recently?
5. How do you monitor logs?
6. How do you check running processes?
7. How do you check whether a port is listening?
8. How do you check whether a service is running?
9. What are exit codes?
10. What is set -e?
11. What is set -x?
12. What is set -u?

Practical / Scenario-Based

1. Write a disk-space monitoring script.

2. Write a service health-check script.

3. Find the top 5 largest files.

4. Search errors in application logs.

5. Count errors in a log file.

6. Restart a service if it is down.

7. Check whether a port is available.

8. Create a backup script.

9. Schedule a backup using cron.

10. Write a script to monitor application health.

Interview Tip: For DevOps Engineer, SRE, and Cloud Engineer roles, prioritize practical questions. Interviewers often give a small Linux problem and ask you to write or explain a script rather than asking only definitions.